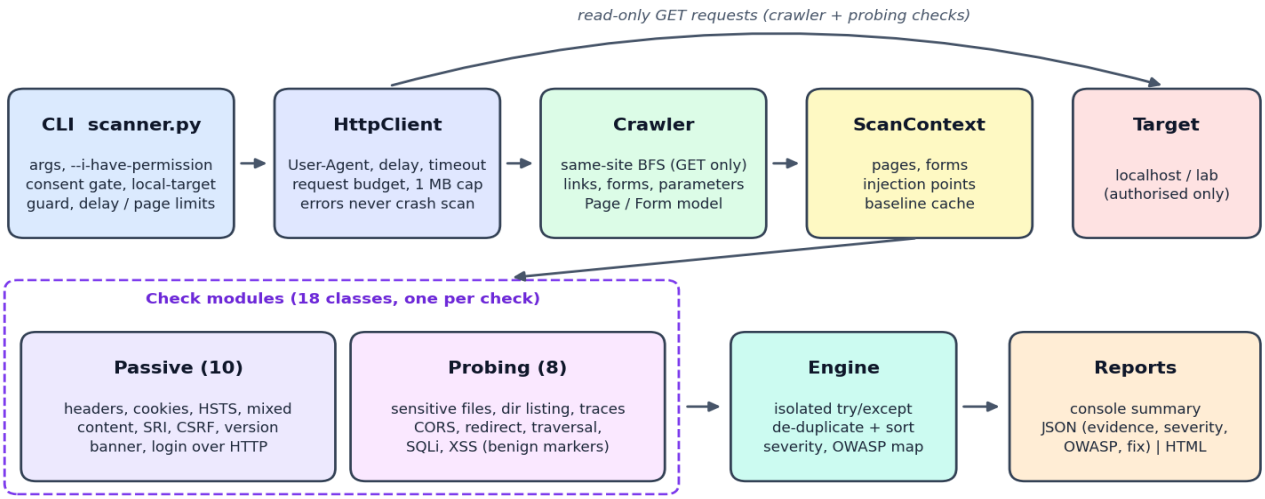
Lightweight Web Vulnerability Scanner

Web / Application Security | Team: [Student name(s) / roll no(s)] | Python 3, requests, beautifulsoup4

1. Overview and ethics

vulnscan is a command-line scanner that crawls one website, runs **18 modular checks** covering **7 OWASP Top 10 (2021) categories** (A01, A02, A03, A05, A06, A07, A08) and reports, for every finding, the evidence, severity, OWASP category and a recommended fix, as a console summary and a JSON report (plus an HTML report). It is **non-destructive**: only GET requests (POST forms are skipped unless *--allow-post*), harmless markers, no data extraction. Safety controls: mandatory *--i-have-permission* gate; refusal of non-local targets unless *--allow-remote*; *--delay* (default 0.5 s), page limit, depth limit, hard request cap, per-request timeout, 1 MB body cap and an explicit User-Agent. Only authorised labs may be scanned (localhost, Juice Shop, DVWA, WebGoat, bWAPP).

2. Architecture



Flow. The CLI validates consent and scope, builds the *HttpClient* and runs the *Crawler* (same-host breadth-first, deduplicating URLs by path + parameter names). Pages, forms and unique injection points are held in a *ScanContext*. Each check is one class (`checks/*.py`) with `run(ctx) -> [Finding]`; the engine runs them in isolation (an exception in one check is logged and the scan continues), de-duplicates findings on (check, title, path, parameter), sorts by severity and writes the reports. Adding a check = adding a class to the registry.

3. Check-to-OWASP mapping

Check ID	OWASP 2021	Type	Technique / false-positive guard	Severity
A01-CORS	A01	Probe	Origin reflected / wildcard / with credentials	Low-High
A01-CSRF	A01	Passive	POST form lacking token-like hidden field	Medium
A01-REDIR	A01	Probe	external canary in redirect params; Location checked, never followed	Medium
A01-TRAV	A01	Probe	../ in file-like params; /etc/passwd or win.ini signature	High
A02-COOKIE-SECURE	A02	Passive	Set-Cookie without Secure	Low-Med
A02-HSTS	A02	Passive	No / short Strict-Transport-Security on HTTPS	Medium
A02-HTTP	A02	Passive	URL scheme is http	Medium
A02-MIXED	A02	Passive	http:// sub-resources on https pages	Low-Med
A03-SQLI	A03	Probe	' and " ; DB error regex absent from baseline	High
A03-XSS	A03	Probe	unique marker + <b> reflected unescaped in HTML	High
A05-DIRLIST	A05	Probe	'Index of /' signature on crawled/common dirs	Medium
A05-FILES	A05	Probe	13 paths; content signature + soft-404 hash + non-HTML	Med-Crit
A05-HEADERS	A05	Passive	CSP, X-Frame-Options/frame-ancestors, nosniff, Referrer-Policy absent	Low-Med
A05-TRACE	A05	Probe	9 framework error signatures (traceback, PHP, .NET...)	Medium
A06-VERSION	A06	Passive	Server / X-Powered-By version + local CVE hints	Low-Med

Check ID	OWASP 2021	Type	Technique / false-positive guard	Severity
A07-COOKIE-FLAGS	A07	Passive	Set-Cookie without HttpOnly / SameSite	Low-Med
A07-LOGIN-HTTP	A07	Passive	password input on non-HTTPS form	High
A08-SRI	A08	Passive	3rd-party script/style without integrity=	Low-Med
(not tested)	A04, A09, A10	Discussion	See section 6: not safely/meaningfully testable by a remote read-only scanner	-

4. Scan results on two lab targets

Targets are the two intentionally vulnerable/hardened lab apps bundled in `labs/` (**VulnLab**: deliberately vulnerable; **HardenedLab**: well configured, but returns HTTP 200 + an HTML shell for *every* unknown path like a single-page app, to provoke false positives). Both run on 127.0.0.1. Reports: `samples/report_vulnlab.json`, `samples/report_hardenedlab.json`. Reproduce with `python tools/evaluate.py`.

Target	Pages	Requests	Time	Findings	Crit / High / Med / Low	OWASP categories hit
VulnLab :8081	14	64	3.16 s	27	1 / 6 / 14 / 6	A01, A02, A03, A05, A06, A07, A08
HardenedLab :8082	6	50	2.46 s	3	0 / 0 / 1 / 2	A01, A02, A05

Severity	Finding (VulnLab)	Where	OWASP
Critical	Exposed Environment file with secrets	/.env	A05
High	CORS reflects arbitrary Origin with credentials	/api/data	A01
High	Path traversal indicator (OS file signature in response)	/view [file]	A01
High	Possible SQL injection (MySQL error message)	/product [id]	A03
High	Reflected XSS: input echoed without HTML encoding	/search [q]	A03
High	Exposed Git repository metadata	/.git/HEAD	A05
High	Login form transmits credentials over HTTP	/login	A07

HardenedLab reported only its three genuine weaknesses: HTTP instead of HTTPS (A02), wildcard CORS on a public endpoint (A01, Low) and a missing Referrer-Policy header (A05, Low). The remaining 20 VulnLab findings are Medium/Low misconfigurations (missing headers, cookie flags, version banners, SRI, CSRF, traceback, directory listing, phpinfo).

5. False positive / false negative analysis

Target	Planted / real issues	True positives	False positives	False negatives	Precision	Recall
VulnLab	30	27	0	3 (by design)	1.0	0.9
HardenedLab	3	3	0	0	1.0	1.0

Important caveat: the labs were written together with the scanner, so these numbers are an *upper bound* that shows the checks work as designed; they are not an estimate of real-world accuracy. Running the tool on Juice Shop, DVWA and an OWASP ZAP baseline scan is the recommended next validation step.

False-positive controls built in:

- **Content signatures, not status codes:** .git/HEAD must start with *ref: refs/*, .env must contain KEY=VALUE lines, phpinfo must contain *PHP Version*, SQL errors must match DB-specific regexes.
- **Soft-404 baseline:** a random non-existent path is fetched first; identical bodies (SPA catch-all) are discarded, and text files that come back as HTML are ignored. HardenedLab (200 for every path) therefore yields 0 false positives.
- **Baseline comparison:** SQLi/traversal findings require the signature to appear *only* after the payload; XSS requires the unique marker plus un-encoded <*b*> in an HTML content type (JSON reflection is ignored; encoded reflection is safe).
- **Open redirect** is confirmed only when the *Location* header points to the canary host; the redirect is never followed. **Deduplication** collapses repeats per (check, title, path, parameter).

False negatives (expected): VulnLab contains three flaws the scanner misses by design: an **IDOR** (/order?id=), a **blind boolean SQL injection** (no error text) and a **DOM-based XSS** (client-side sink). Others: JavaScript-rendered pages (static crawler), POST-only parameters (read-only mode), custom error pages without known signatures, parameters named unusually for traversal/redirect checks, anything behind a login. Compared with an OWASP ZAP baseline scan, a lightweight scanner will not match ZAP's AJAX spider, passive rule set breadth or active scan depth, and cannot find business-logic flaws, blind/time-based injection, SSRF or access-control bugs behind

authentication.

6. Limitations (including A04, A09, A10)

- **A09 Logging and Monitoring Failures:** logging, alerting and incident response occur server-side and leave no signal a remote read-only client can observe. Assess by reviewing log configuration, retention and alert rules.
- **A10 SSRF:** confirming SSRF safely needs an out-of-band callback server or access to internal services; sending such payloads would risk harming third parties, so it is intentionally excluded. **A04 Insecure Design** needs threat modelling, not probing.
- No authentication, no JavaScript execution, GET-only by default, single host scope, 1 MB body cap, and signature matching can both miss custom messages and flag pages that merely quote error text. HTTPS-only checks (HSTS, mixed content, Secure on HTTPS) are covered by unit tests with fabricated HTTPS pages because the labs run over HTTP. Version-to-CVE hints use a tiny offline table and must be verified because distributions back-port fixes.

7. Remediation advice

Issue	Recommended fix
SQL injection (A03)	Parameterised queries / ORM bind variables; validate types; least-privilege DB account; generic error pages.
Reflected XSS (A03)	Context-aware output encoding, auto-escaping templates, strict CSP (no unsafe-inline), HttpOnly cookies.
Path traversal / open redirect (A01)	Map IDs to files via allow-list and canonicalise paths; redirect only to relative paths or allow-listed hosts.
CORS / CSRF (A01)	Allow-list origins, never reflect Origin with credentials; per-session CSRF tokens + SameSite=Lax/Strict.
No HTTPS / HSTS / cookie flags (A02, A07)	TLS 1.2+ everywhere, 301 to HTTPS, HSTS max-age>=1 year; Secure, HttpOnly, SameSite on session cookies.
Exposed .git/.env/phpinfo/listing/traces (A05)	Deny dotfiles and backups in the web server, remove debug pages, Options -Indexes, disable debug mode, rotate leaked secrets.
Missing security headers (A05)	CSP, X-Frame-Options or frame-ancestors, X-Content-Type-Options: nosniff, Referrer-Policy.
Version banners (A06)	ServerTokens Prod / server_tokens off / expose_php=Off; patch or upgrade EOL components; track CVEs with an SCA tool.
Missing SRI (A08)	Add integrity="sha384-..." + crossorigin, pin versions or self-host third-party scripts.

Bonus items implemented: HTML report with severity chart (*--html*), version-to-CVE lookup from local data, CI step failing on High findings (*--fail-on high*, GitHub Actions workflow). **Tests:** 32 pytest tests (unit tests for every check with fabricated pages, integration tests against the labs, CLI/consent-gate tests).